

Perception MEA4



1. Introduction :

Dans le cadre de ce projet, nous allons découvrir comment détecter une balle en temps réel dans un environnement, à l'aide de la bibliothèque OpenCV. L'ensemble des codes seront développés en langage Python. Le travail est découpé en plusieurs parties. On commencera par travailler sur une image. Puis une fois une méthode fiable trouvée, nous la testerons sur plusieurs autres images. Enfin, nous adapterons le code pour faire de la détection dans une vidéo avant de détecter la balle en temps réel, à partir de n'importe quelle caméra.

On ne considèrera que les balles de couleurs jaunes tout au long de ce projet.

L'ensemble des codes et des images utilisés lors de ce projet sont disponibles ici :

https://github.com/ben-team/OpenCV_Detection_Balle_Jaune.git

2. Etude et traitement sur une image statique

a) Détection de la couleur jaune sur une image en RGB :

Chaque pixel de l'image est représenté en mode RGB (R = Rouge, G = Vert et B = Bleu). On cherche une balle de couleur jaune. L'idée est de parcourir chaque pixel de l'image et de sélectionner ceux dont la couleur est proche de celle de la balle.

Cependant, il existe de nombreuses variations de la couleur jaune en fonction de l'intensité de Bleu, Vert et Rouge. On définit alors une couleur de référence R_0 , G_0 , B_0 . Ce pixel de référence est défini en sélectionnant un pixel de la balle sur l'image. Puis on compare l'ensemble des pixels de l'image à ce pixel de référence :

$$|(R, G, B) - (R_0, G_0, B_0)| < \epsilon$$

Epsilon correspond à la marge d'erreur entre le pixel que l'on compare et le pixel de référence.

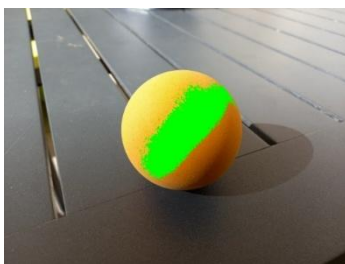


Figure 1 : Pixel de référence au centre de la balle. Epsilon fixé à 80.

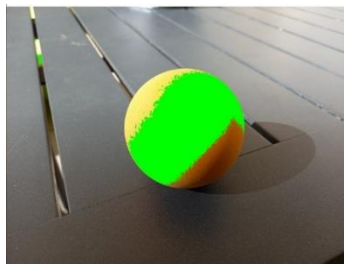


Figure 2 : Pixel de référence au centre de la balle. Epsilon fixé à 100.

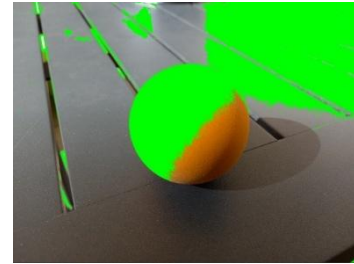


Figure 3 : Pixel de référence sur la partie supérieure de la balle. Epsilon fixé à 100.

Remarque : Le pixel de référence a une grande influence sur la détection. En effet d'après les figures 2 et 3, on remarque qu'avec un même epsilon mais un pixel de référence différent les pixels sélectionnés sont complètement différents. Sur la figure 2, on ne détecte que la moitié des pixels de la balle, alors que sur la figure 3, on détecte l'environnement et la moitié des pixels de la balle. Epsilon a également une grande influence sur la détection. En effet d'après les figures 1 et

2, on remarque qu'avec un même pixel de référence mais un epsilon différent, on détecte soit la moitié des pixels de la balle, soit presque rien.

On en conclut que la détection dans ce mode de couleur est très dépendante de l'epsilon et du pixel de référence. Après de nombreux tests, les résultats ne sont pas concluants avec cette méthode.

b) Détection de la couleur jaune sur une image en HSV sans V :

Le mode HSV (H=Teinte, S=Saturation, V=Valeur) est un domaine colorimétrique où V représente les ombres. On réapplique le traitement d'image réalisé à la partie (a) c'est-à-dire qu'on parcourt chaque pixel de l'image et on sélectionne ceux dont la couleur est proche de celle de la balle, mais cette fois ci en HSV, sans prendre en compte les ombres. En effet, quand on ne prend pas en compte les ombres on réduit les variations de la couleur jaune possibles. Ainsi, on définit un pixel de référence (H_0, S_0) et on obtient le résultat présenté sur les Figures 4 à 6. J'ai choisi d'illustrer la sélection des pixels dans le mode RGB pour faciliter la visualisation.

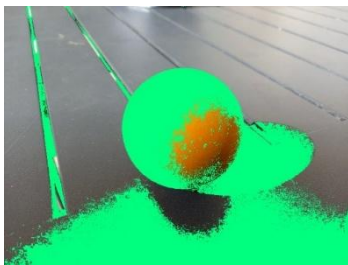


Figure 4 : Pixel de référence en haut de la balle (zone claire). Epsilon fixé à 100.

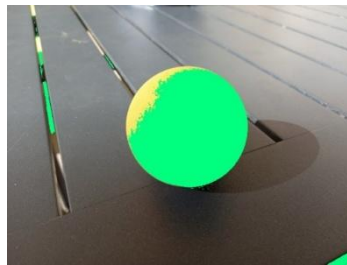


Figure 5 : Pixel de référence en bas de la balle. Epsilon fixé à 100.

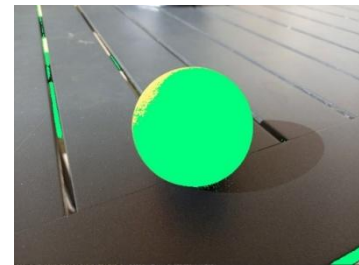


Figure 6 : Pixel de référence au milieu de la balle. Epsilon fixé à 100.

Visualisation en RGB de la détection

On remarque que pour une même valeur d'epsilon et un pixel de référence différent on obtient d'après la figure 5 et 6 une détection équivalente. D'après la figure 4 on remarque qu'un pixel sélectionné sur une zone claire de la balle ne permet pas de sélectionner les pixels de la balle.

On en conclut que la représentation en HSV permet de détecter la balle plus facilement qu'en mode RGB car ce domaine de couleur est moins dépendant d'une des deux variables. En effet, dans de nombreux cas, en posant Epsilon, on peut détecter la balle en faisant varier uniquement le pixel de référence.

c) Détection de la couleur jaune sur une image en HSV sans S et sans V :

On obtient de meilleurs résultats dans le domaine HSV quand on ne prend pas en compte le paramètre V. On va essayer d'appliquer le même traitement d'image sans la composante S pour être sûr qu'on n'obtient pas de meilleurs résultats. On définit un pixel de référence (H_0) et un epsilon. On obtient le résultat suivant :



Figure 7 : Pixel de référence pris en bas de la balle. Epsilon fixé à 5.

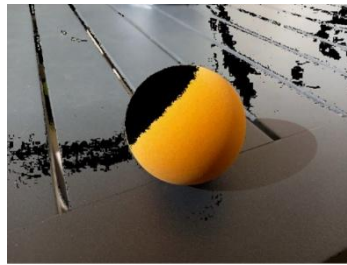


Figure 8 : Pixel de référence pris en haut de la balle. Epsilon fixé à 5.

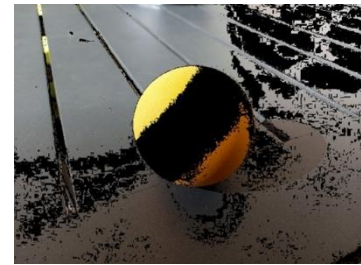


Figure 9 : Pixel de référence pris au milieu de la balle. Epsilon fixé à 5.

On conclut qu'il est impossible de détecter la balle avec uniquement la composante H (la teinte). En effet, même avec un epsilon très petit, l'algorithme détecte des pixels similaires partout sur l'image.

Finalement, on en conclut que la représentation des couleurs en HSV est la plus appropriée pour détecter une couleur dans un environnement. En revanche, elle ne suffit pas pour détecter un objet car on ne fait que trouver les pixels dans une gamme de couleur mais on ne définit pas la forme. De plus, ce traitement dépend de la sélection d'un pixel de référence, or dans le cadre de la détection de la balle dans une vidéo, on ne souhaite pas avoir à définir de pixel de référence.

d) Détection de la couleur jaune sur une image avec un masque :

Un masque permet de parcourir chaque pixel de l'image et de sélectionner ceux qui font partie d'une plage de valeurs prédéfinies. L'avantage d'un masque est que le parcours des pixels est réalisé en C plutôt qu'en Python, ce qui est beaucoup plus rapide.

Ainsi on définit la plage délimitée par les deux traits violets qui permet de sélectionner un grand nombre de variations de la couleur jaune en HSV :



Figure 10 : Visualisation de la plage de couleur du masque

Avant d'appliquer le masque, on applique à chaque pixel de l'image un $V=255$ pour obtenir une image où les couleurs sont très éclatantes d'après la figure 11. Ensuite on lisse l'image avec les fonctions « Erode » puis « Dilate » pour détecter plus facilement les zones de couleurs.

Pour visualiser l'effet de ce traitement, on reporte sur la figure 12 l'image dans le mode RGB après les deux transformations. Le passage en RGB n'est que pour visualiser le résultat, tout le traitement est fait en HSV.

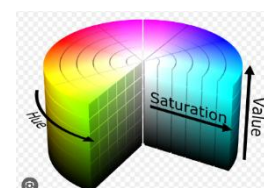


Figure 11 : Visualisation du domaine de couleur HSV

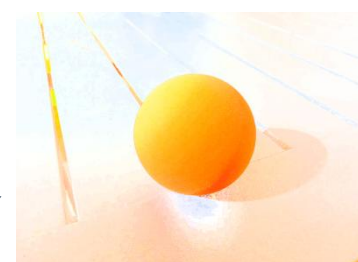


Figure 12 : Visualisation des deux transformations en mode RGB

Enfin, on applique le masque à l'image traitée en HSV pour ne garder que les couleurs qui nous intéressent. On obtient la figure 13.



Figure 13 : Visualisation de l'image en mode HSV obtenue après application du masque

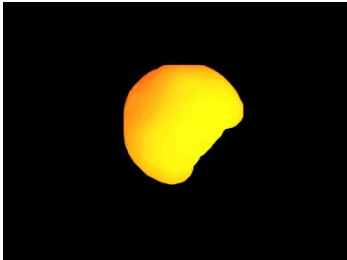


Figure 14 : Visualisation de l'image en mode HSV obtenue après application du masque et du lissage

On applique à nouveau les fonctions "Erode", puis "Dilate", à la figure 13 pour enlever les impuretés (en rouge sur la figure 13) et on obtient la figure 14.

Maintenant que nous avons réussi à extraire les pixels de l'image sans impureté et sans avoir à définir de pixel de référence nous devons appliquer un filtre supplémentaire pour détecter si l'objet peut être une balle ou pas. L'objet est une balle s'il a la forme d'un cercle sinon c'est autre chose.

f) Détection de la forme d'une balle :

Pour étudier la forme de la balle il existe la méthode de HoughCircle qui permet sur une image en niveau de gris de détecter des cercles. Pour cela, on passe l'image en niveau de gris puis on utilise la fonction « threshold » pour uniformiser les couleurs. Cela permet de réduire les erreurs de détection de cercle dues au niveau de gris non uniforme.

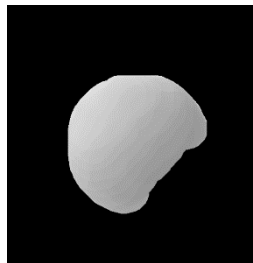


Figure 15 : Visualisation de l'image en niveau de gris

On applique l'algorithme de HoughCircle sur l'image en niveau de gris, puis on trace le cercle obtenu sur l'image en mode RGB. On obtient le résultat suivant :

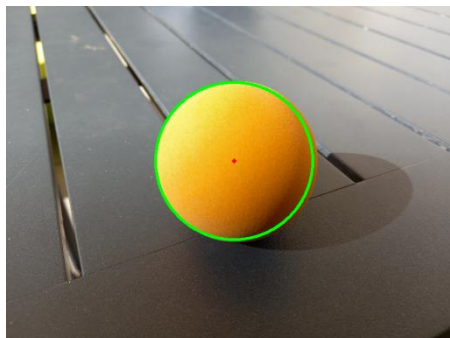


Figure 16 : Visualisation de la détection de la balle sur l'image en RGB

On en conclut que pour détecter un objet sur une image il faut obligatoirement travailler à la fois sur la couleur et sur la forme. Au niveau de l'espace colorimétrique, l'espace HSV est le plus adapté au traitement de l'image.

g) Traitement sur d'autres images :

On peut vérifier l'efficacité de l'algorithme sur plusieurs autres images :



Figure 17 : Détection de plusieurs balles jaunes dans un environnement rempli de balles



Figure 18 : Détection d'une balle jaune dans une main



Figure 19 : Détection d'une balle jaune dans un environnement simple avec quelques ombres

On remarque que les balles jaunes sont plutôt bien détectées. De plus l'algorithme permet de détecter plusieurs balles jaunes dans une même image.

3. Traitement sur une vidéo et sur un « live »

Une vidéo est composée de millions d'images mises les unes à la suite des autres. La première idée est d'analyser chaque image pour en déduire où est la balle. Ainsi on applique l'algorithme de traitement avec le masque à chaque image.

En appliquant l'algorithme on constate que la vidéo est fortement ralentie, on a environ deux images par seconde. Le traitement sur chaque image est lourd et prend du temps.

Dans le domaine des jeux vidéo, le DLSS (Deep Learning Super Sampling) développé par NVIDIA permet de créer une image dans une définition inférieure, puis de l'agrandir avec l'IA à une résolution supérieure. Ici, nous allons reprendre la même idée mais dans l'autre sens. C'est-à-dire que nous allons réduire la taille de l'image d'un certain facteur puis nous ferons tout le traitement de la reconnaissance de la balle sur cette nouvelle image mais de taille plus petite. Une fois le traitement réalisé, le cercle de détection sera ensuite tracé sur l'image de taille initiale. Cette méthode permet de faire les calculs sur un nombre de pixels inférieur et donc de gagner du temps pour garder le flux vidéo.

On teste ensuite plusieurs facteurs de réduction pour trouver un compromis entre fluidité (FPS = Frame Per second) et efficacité de la détection. On constate un problème : si le traitement est trop rapide, la vidéo est accélérée par rapport à la vidéo originale. Pour s'assurer d'avoir une vidéo aussi fluide que l'originale, on mesure le temps d'exécution du traitement de l'image, connaissant le nombre d'images par seconde de la vidéo originale on fait attendre l'algorithme le temps nécessaire pour garder le FPS de la vidéo originale.

La vidéo originale utilisée dans ce projet est cadencée à 28 FPS. Lorsque l'on divise par deux le nombre de pixels, on obtient 18 FPS et la balle est détectée correctement. Lorsque l'on divise par trois le nombre de pixels, on obtient 20 FPS et la balle est détectée correctement. Dans ces deux

derniers cas, la vidéo est ralentie par rapport à la vidéo initiale, mon code ne rajoute pas de retard puisque le temps de calcul est déjà trop long. Lorsque l'on divise par quatre le nombre de pixels, on obtient 28 FPS et la balle est détectée également correctement. Dans ce cas, mon code rajoute un retard pour s'adapter au FPS de la vidéo originale. Donc pour avoir une vidéo aussi fluide que la vidéo originale il faut un facteur de dimension égale à 4. On pourrait penser qu'utiliser un facteur de dimension supérieur, par exemple égale à 6, pourrait accélérer encore plus les calculs. Malheureusement pour un facteur 6, il n'y a plus assez de pixels pour détecter correctement la balle. Finalement pour détecter une balle sur une vidéo il faut trouver un compromis entre précision et fluidité.

On peut à présent faire de la détection en temps réel. Je me suis appuyé sur le même algorithme que sur la vidéo. En revanche on ne calcul plus le retard de l'algorithme par rapport au flux vidéo. En effet, le retard est inclus naturellement dans le flux vidéo. L'algorithme ne peut pas traiter la prochaine image tant qu'il ne l'a pas reçue.

4. Conclusion :

Finalement ce projet nous a permis d'appréhender la difficulté de détecter un objet dans un environnement, en utilisant différentes méthodes de traitement d'image. Nous avons appris à utiliser différentes méthodes de perception d'environnement ainsi qu'un outil très puissant : OPENCV.